

STANDARD OPERATING PROCEDURE (SOP)

Engineering Coordination Center for Frappe Framework & ERPNext (Frappe ECC)

Document ID	SOP-ENG-FRAPPE-ECC-001
Version & Release	Version 1.1.0 (Full 20-Agent Engineering Release)
Effective Date	2026-09-18
Applicability	Frappe v14 / v15 / v16, ERPNext, Antigravity, Claude Code, Cursor, Codex

1. Purpose & Scope

This Standard Operating Procedure defines the mandatory engineering practices, execution phases, and automated quality controls for building, testing, and securing Frappe Framework applications using Frappe ECC. Frappe ECC equips your AI assistant with 20 specialist agents, 18 production skills, 20 slash commands, and Frappe Shield.

2. System Architecture & Inventory

Component Layer	Count	Core Responsibilities
Specialist AI Agents	20	HLD, LLD, UI/UX, Wireframes, Clickable Prototypes, Fullstack Turnkey Dev, Manual QA, Automated Testing (Playwright), Planning, Controllers, Desk UI, TDD, Code Review, Security, DevOps, Migrations, Reporting, APIs, Docs
Workflow Skills	18	HLD/LLD design, Wireframing & Prototyping, QA testing & Playwright automation, Turnkey scaffolding, DocType modeling, QueryBuilder, hooks.py, Desk scripts, REST API, Permissions, TDD, background jobs, reports, patches, Bench CLI, Frappe UI, Portal, Security Audit
Slash Commands	20	/frappe:hld, /frappe:lld, /frappe:wireframe, /frappe:prototype, /frappe:build-e2e, /frappe:manual-qa, /frappe:e2e-test, /frappe:plan, /frappe:doctype, /frappe:controller, /frappe:client-script, /frappe:hook, /frappe:api, /frappe:test, /frappe:review, /frappe:security,

		/frappe:patch, /frappe:report, /frappe:bench, /frappe:help
Coding Rules	5	Always-loaded standards: Core architecture, Python backend, Desk JS, Security/Permissions, and Database optimization
Frappe Shield	1 CLI	AST static analyzer detecting SQL injection in frappe.db.sql, db.commit() violations, unvalidated guest APIs, and N+1 loops

3. Installation Procedure

[SETUP INSTRUCTION] Antigravity setup registers all 18 skills into your global config directory so every workspace automatically accesses Frappe domain intelligence.

Command for Linux / macOS / WSL:

```
./install.sh --profile full --target antigravity
```

Command for Windows PowerShell:

```
.\install.ps1 -Profile full -Target antigravity
```

4. The 12-Phase Feature Development SOP

Phase 1: High-Level Design (HLD) & System Context

Author system context, component topologies, and non-functional requirements (SLA, caching with Redis, multi-tenant bench topology) using '/frappe:hld'.

Phase 2: Low-Level Design (LLD) & Entity-Relationship Modeling

Generate precise entity-relationship diagrams (Mermaid erDiagram), class hierarchies, and state transition tables using '/frappe:lld'.

Phase 3: Visual Wireframing & Clickable Interactive Prototyping

Create visual ASCII/SVG wireframes using '/frappe:wireframe', followed by standalone clickable HTML/Vue prototypes using '/frappe:prototype' for stakeholder approval before code is written.

Phase 4: Schema Modeling & DocType Scaffolding

Generate schema JSONs with sections, column breaks, indexes, and role permission arrays using `/frappe:doctype`.

Phase 5: Test-Driven Development (TDD) — Unit Tests First

Draft failing test assertions in `'test_<doctype>.py'` using `FrappeTestCase`. Confirm tests fail (RED) before implementing controllers.

Phase 6: Turnkey Fullstack Implementation (Zero Placeholders)

Synthesize complete vertical implementations (controller lifecycle, Desk client scripts, dialogs, auto-math) using `/frappe:build-e2e`. Zero placeholders allowed.

Phase 7: Hook Registration & Event Orchestration

Wire document events, background cron jobs, and class overrides into `hooks.py` using `/frappe:hook`.

Phase 8: Whitelisted APIs & Webhook Integrations

Expose secure REST endpoints with role checks (`frappe.only_for`), rate limiting, and input sanitization using `/frappe:api`.

Phase 9: Database Schema Migrations & Patches

Author idempotent migration patches in `'patches/'` and register in `'patches.txt'` using `/frappe:patch`.

Phase 10: Manual QA Matrix & Exploratory Verification

Generate structured manual test scenarios, edge-case boundary tests, and release sign-off checklists using `/frappe:manual-qa`.

Phase 11: End-to-End Automated Browser Testing (Playwright)

Author headless Playwright browser tests that click through Desk forms, fill fields, and assert submission badges using `/frappe:e2e-test`.

Phase 12: Security Audit with Frappe Shield & Code Review

Execute AST static scanning with Frappe Shield (`/frappe:security`) to ensure zero SQL injection, zero manual `db.commit()` calls, and zero security flaws.

5. Master Command Cheat Sheet

Slash Command	Specialist Agent	Standard Operation
---------------	------------------	--------------------

<code>/frappe:hld '<feature>'</code>	frappe-hld-architect	Generate High-Level Design with C4 architecture diagrams
<code>/frappe:lld '<name>'</code>	frappe-lld-designer	Generate Low-Level Design with ER & state machine diagrams
<code>/frappe:wireframe '<name>'</code>	frappe-wireframe-builder	Generate visual ASCII / SVG wireframe layout mockups
<code>/frappe:prototype '<name>'</code>	frappe-interactive-prototyper	Generate clickable interactive HTML/Vue prototype
<code>/frappe:plan '<feature>'</code>	frappe-planner	Architect feature schema blueprint & module taxonomy
<code>/frappe:doctype '<Name>'</code>	frappe-planner	Scaffold .json, .py, .js, and test_*.py files
<code>/frappe:build-e2e '<spec>'</code>	frappe-fullstack-developer	Turnkey fullstack vertical synthesis (zero placeholders)
<code>/frappe:controller '<Name>'</code>	frappe-backend-builder	Implement controller lifecycle hooks and validations
<code>/frappe:client-script '<Name>'</code>	frappe-desk-builder	Build Desk form UI interactions & custom dialogs
<code>/frappe:hook [event cron]</code>	frappe-architect	Wire doc_events, scheduler cron, or class overrides
<code>/frappe:api '<name>'</code>	frappe-api-integrator	Create secure whitelisted REST API endpoint
<code>/frappe:test '<Name>'</code>	frappe-tdd-guide	Scaffold unit tests with FrappeTestCase
<code>/frappe>manual-qa '<Name>'</code>	frappe-manual-qa	Generate manual test scenarios & QA sign-off checklist
<code>/frappe:e2e-test '<Name>'</code>	frappe-automated-tester	Generate Playwright automated browser test script
<code>/frappe:review [path]</code>	frappe-code-reviewer	Review diffs in fresh context for Frappe anti-patterns
<code>/frappe:security [path]</code>	frappe-security-reviewer	Execute AST security scanner (SQLi, IDOR, XSS)
<code>/frappe:patch '<desc>'</code>	frappe-migration-patcher	Draft idempotent database migration patch in patches.txt
<code>/frappe:report '<name>'</code>	frappe-report-builder	Scaffold Script Report (Python backend + JS)

		frontend)
/frappe:bench [task]	frappe-bench-devops	Diagnose bench errors and site operations
/frappe:help	All Agents	Display master quick cheat sheet

APPROVED & ENFORCED BY: Antigravity Systems Engineering Group
DISTRIBUTION: Open Source / All Engineering Teams